Introduction to Computer Architecture and Systems

Chapter 5
Control Flows, Hazards

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

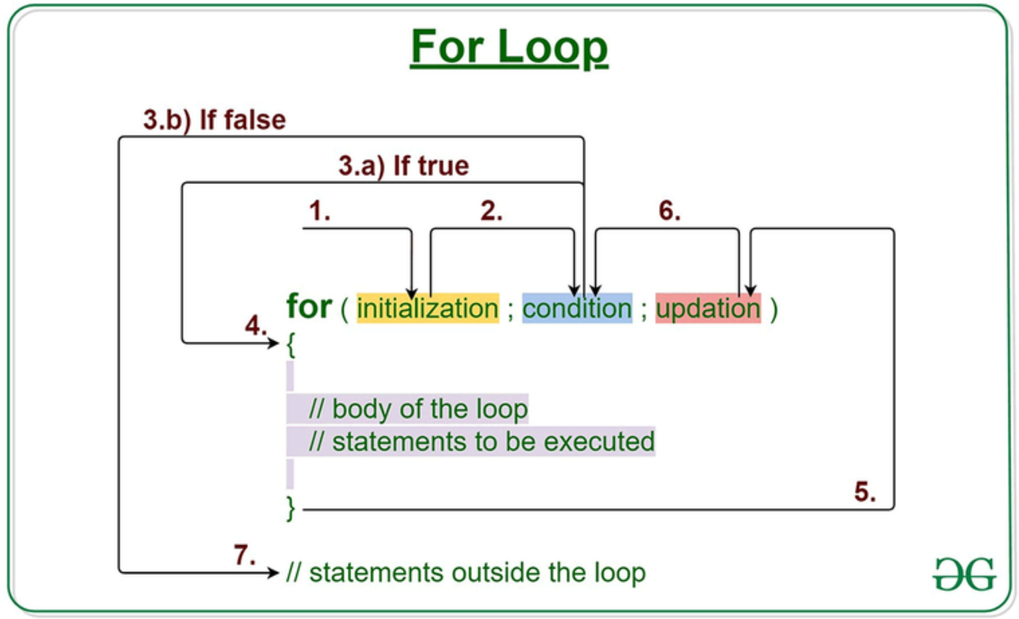
Previously of CSCI 260…

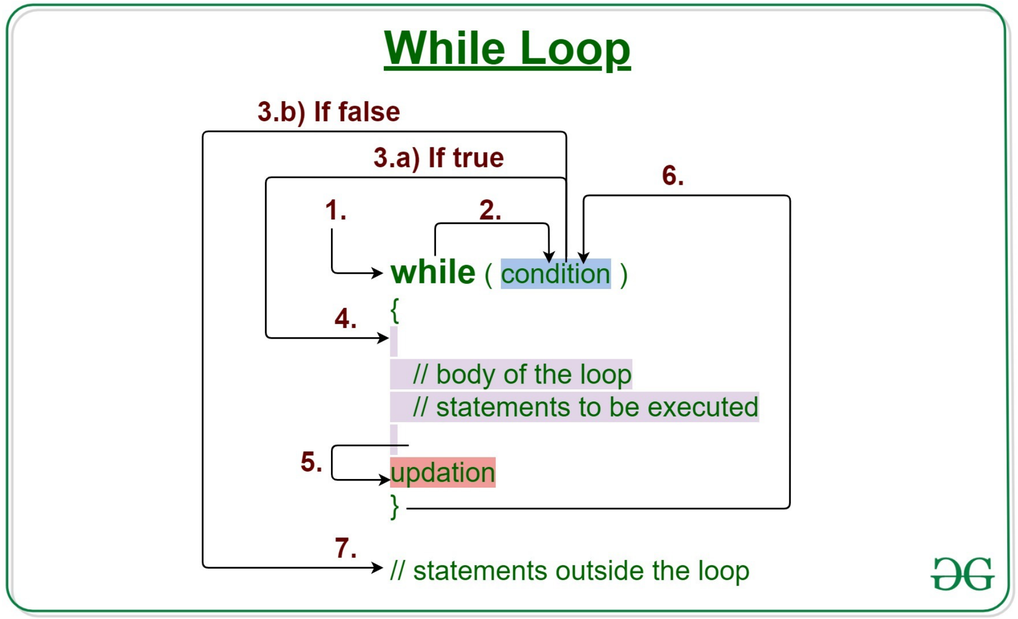
- Essential Tools and Mindset to write in Assembly (and all other languages)
- Programming in LC-3
- CISC and RISC
  (Note that, now all CISC processors are running actually RISC, with compilers.)

3 Essential Elements for a Loop

Iterators, Conditions, and State Shifters…

Sometimes they are not all visible, but they are always there!





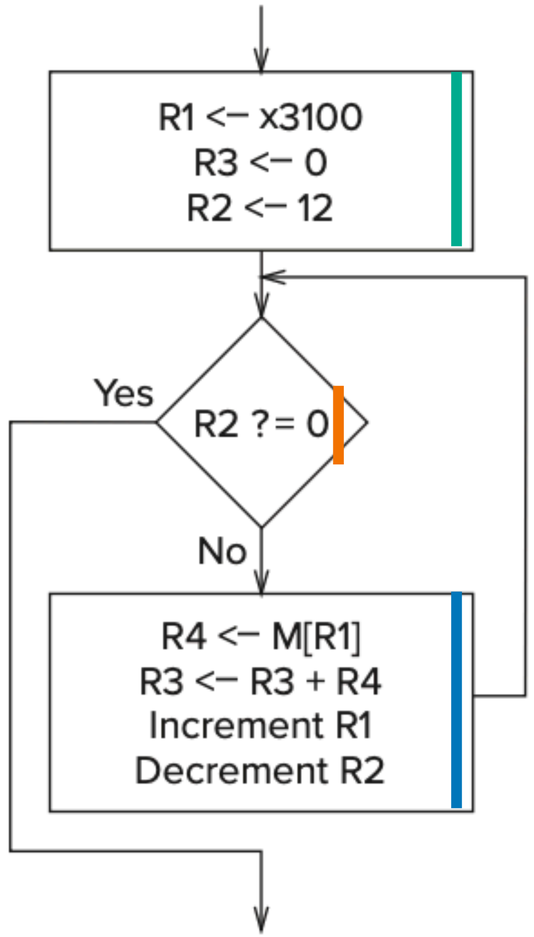
How do these 3 elements represent in Assembly?

Loops…

Two ways to create loops: Counters, and Sentinels.

Example: Add a number n times…

Question: From x3100 to x310B, we have 12 integers, get the sum of them.




```
R1<- 3100
R3 <- 0
R2 <- 0
R2 <- 12
BRz x300A
R4 <- M[R1]
R3 <- R3+R4
R1 <- R1+1
R2 <- R2-1
BRnzp x3004
```

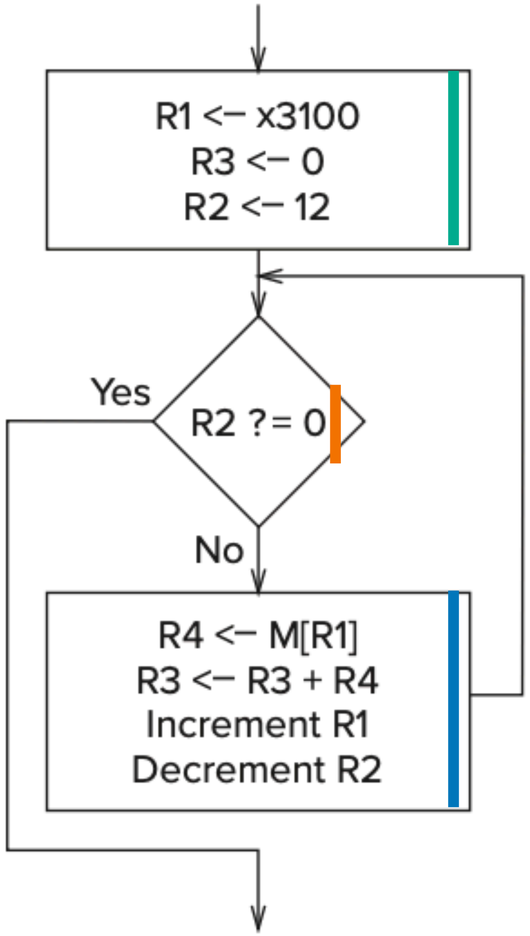
Example: Add a number n times…

Question: From x3100 to x310B, we have 12 integers, get the sum of them.

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	R1<- 3100
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3002	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3003	0	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 <- 12
x3004	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1	BRz x300A
x3005	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 <- M[R1]
x3006	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 <- R3+R4
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1+1
x3008	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3009	0	0	0	0	1	1	1	1	1	1	1	1	1	0	1	0	BRnzp x3004

Example: Add a number n times…

Question: From x3100 to x310B, we have 12 integers, get the sum of them.

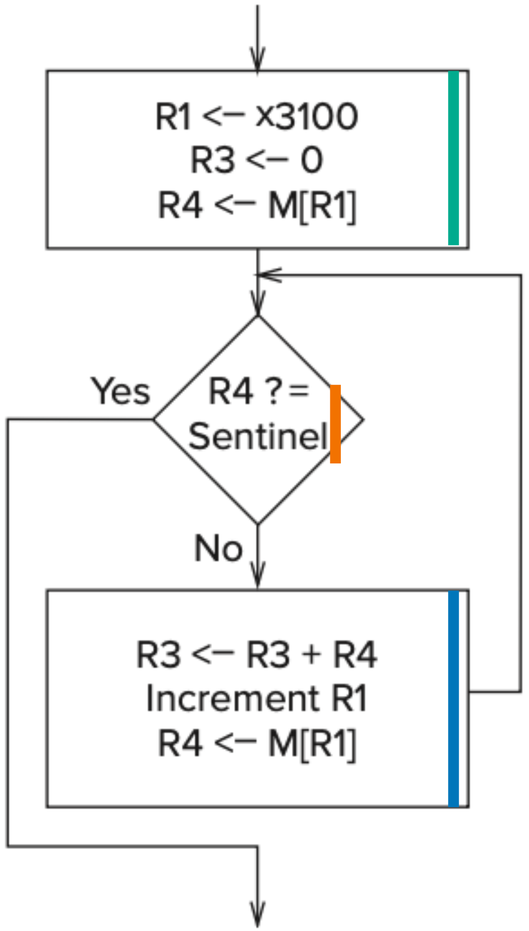


- R2 is a counter.
- By setting up a counter, modify it after successful operation and direct flow by that counter, we can perform loop designated times.

Example: Add a number n times…

Question: From x3100, we have several integers, get the sum of them.

- Instead of setting up a counter, we get some values to indicate the loop should be over…




```
R1<- x3100
R3 <- 0
R4 <- M[R1]
BRn x3008
R3 <- R3+R4
R1 <- R1+1
R4 <- M[R1]
BRnzp x3003
```

Example: Add a number n times…

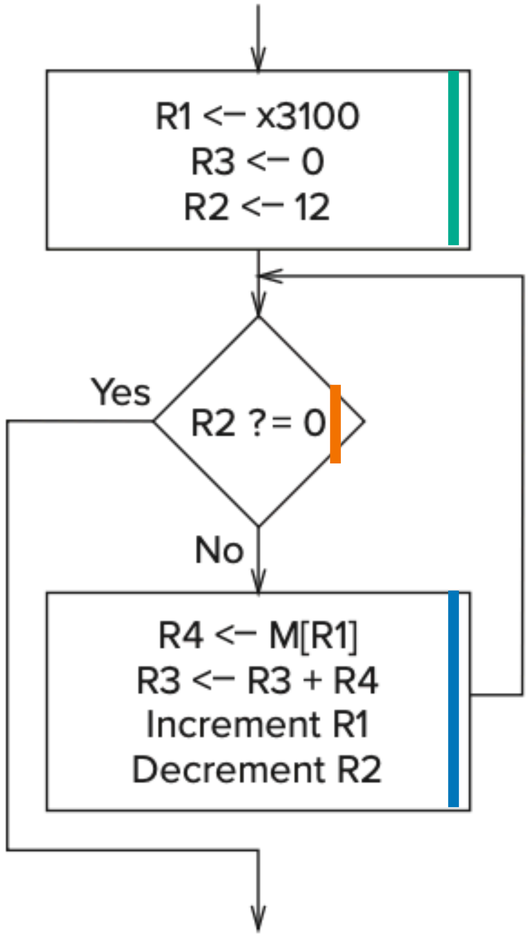
Question: From x3100, we have several integers, get the sum of them.

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0
x3002	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0
x3003	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0
x3004	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0
x3005	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1
x3006	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0
x3007	0	0	0	0	1	1	1	1	1	1	1	1	1	0	1	1

```
R1<- x3100
R3 <- 0
R4 <- M[R1]
BRn x3008
R3 <- R3+R4
R1 <- R1+1
R4 <- M[R1]
BRnzp x3003
```

Example: Add a number n times…

Question: From x3100, we have several integers, get the sum of them.



- R2 .
- By setting up a counter, modify it after successful operation and direct flow by that counter, we can perform loop designated times.

Conclusion

- There are 2 ways to do loops by using
  - A counter, or
  - Sentinel.

Assignment 1

Assignment 1

Fibonacci Sequence

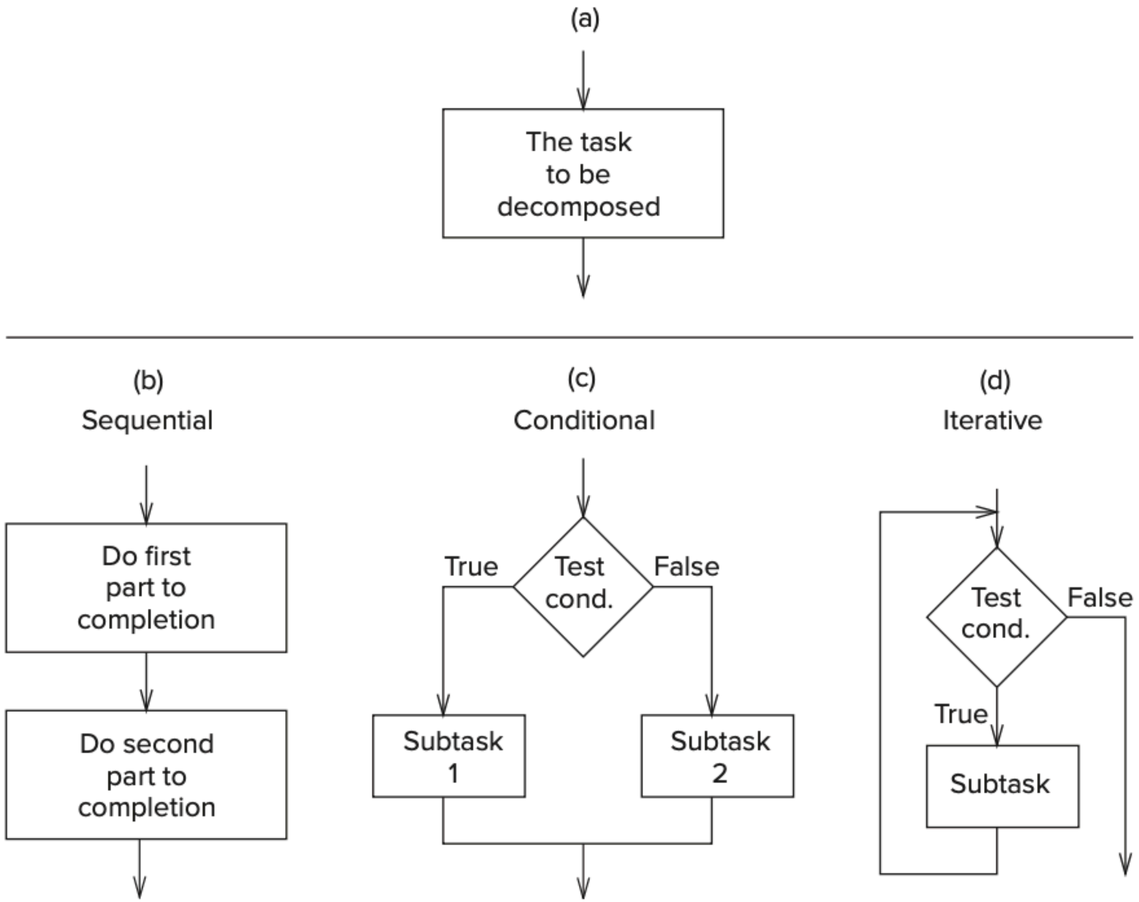
- Write in LC-3 Assembly to **calculate the 20th number of fibonacci sequence; calculate nth number of the sequence if you can.**
- Use 5-step approach to write code, and run it on LC3Tools.
- Submit your code in coding form, and a screenshot of LC3Tools.
- Assignment is releasing tonight, and due 23:50 this Sunday(10/05).
- Seek help with your classmates, and visit my office hours, if in doubt.

Systematic Decomposition

Another Missing Piece…

Control flow when breaking down the problem…

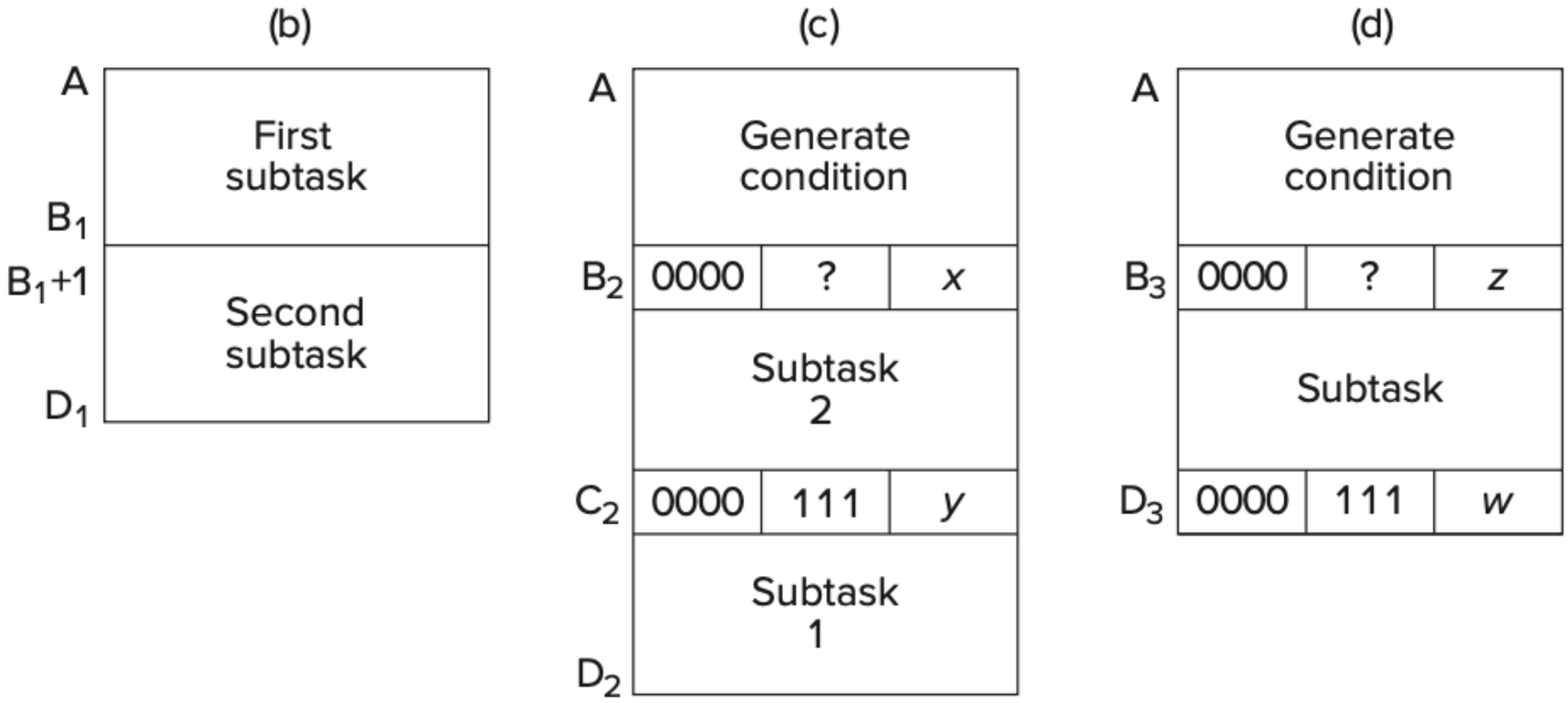
- 3 types of control flow…



Another Missing Piece…

Control flow when breaking down the problem…

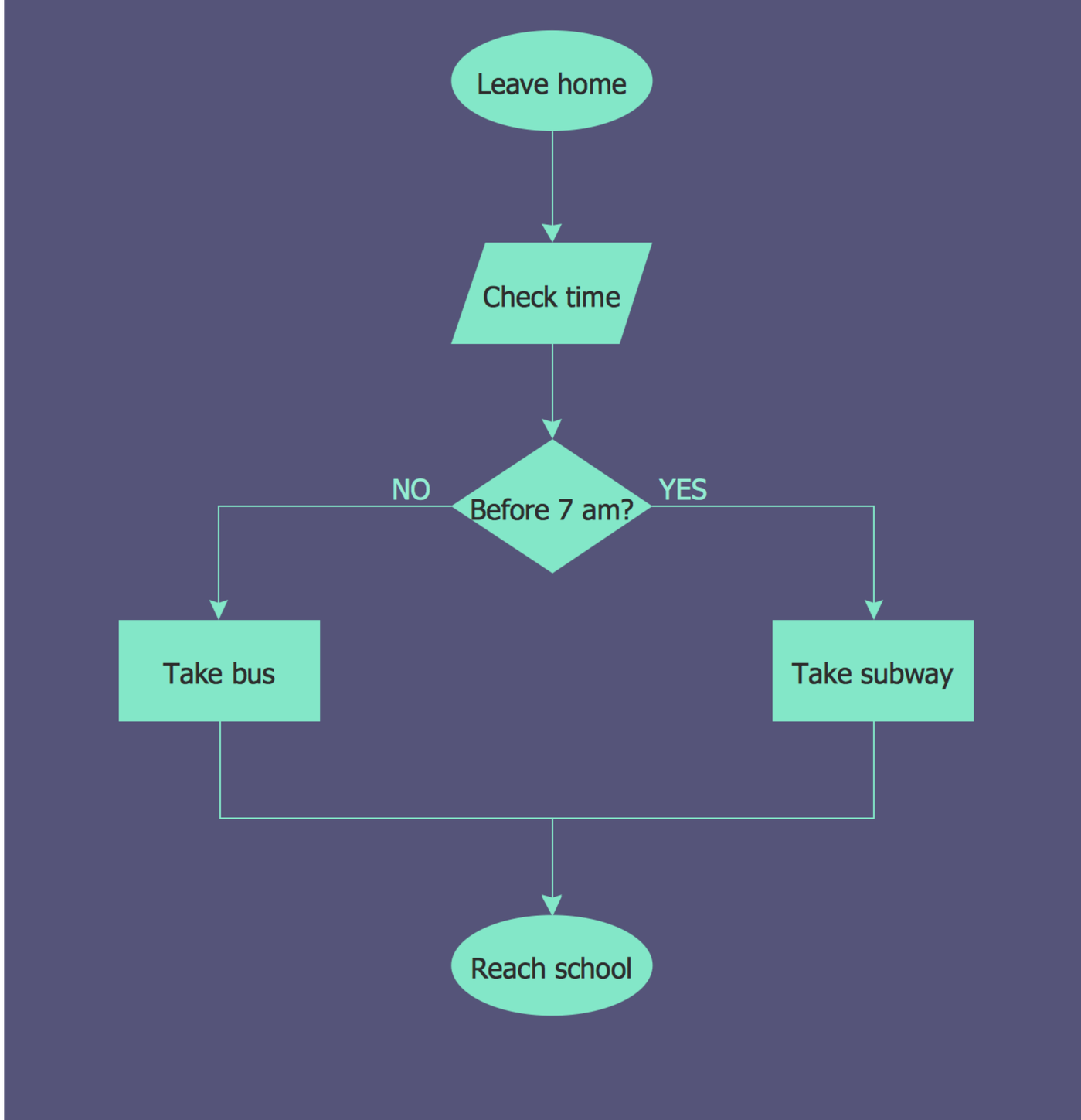
- 3 types of code organization…



Presenting Controlflow

Flowchart

- Circles: Start / End States
- Parallelograms: Input / Output
- Diamonds: Condition
- Rectangles: Actions

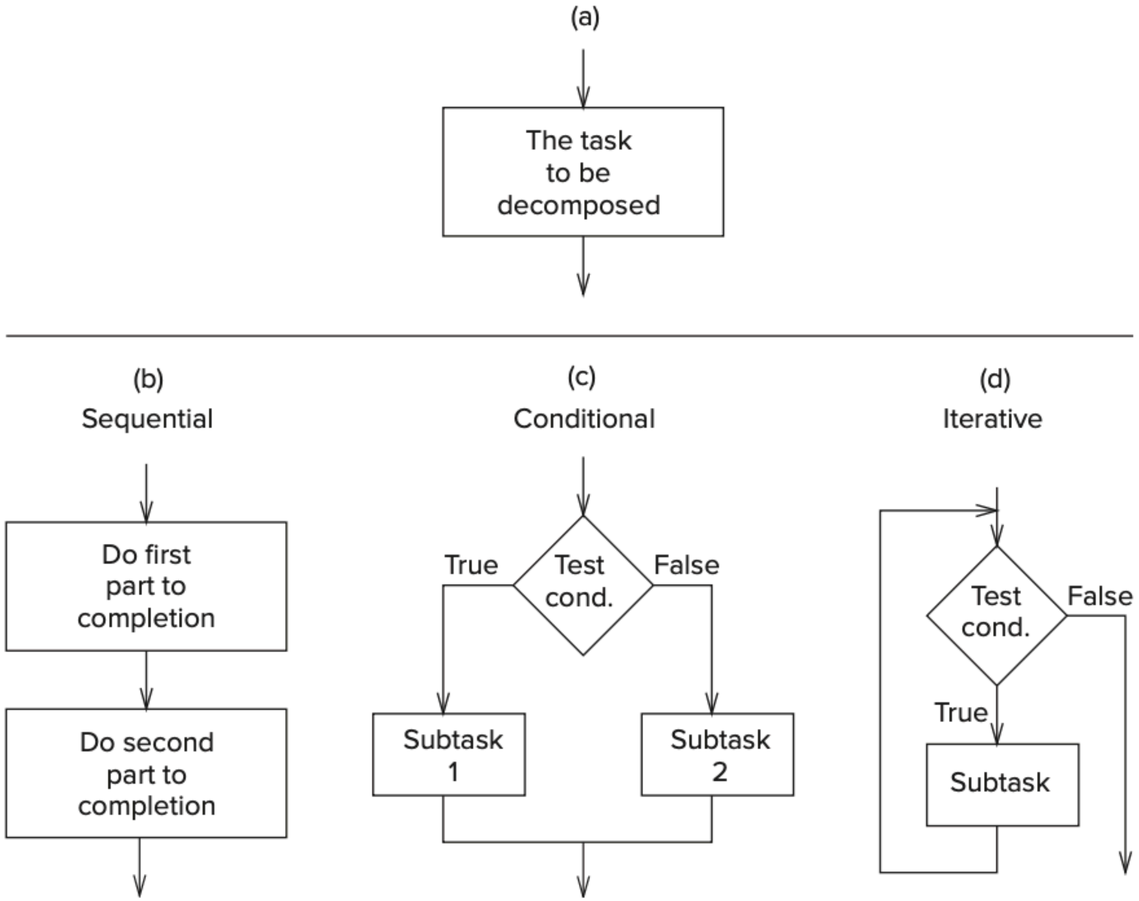


Systematic Decomposition

Another Missing Piece…

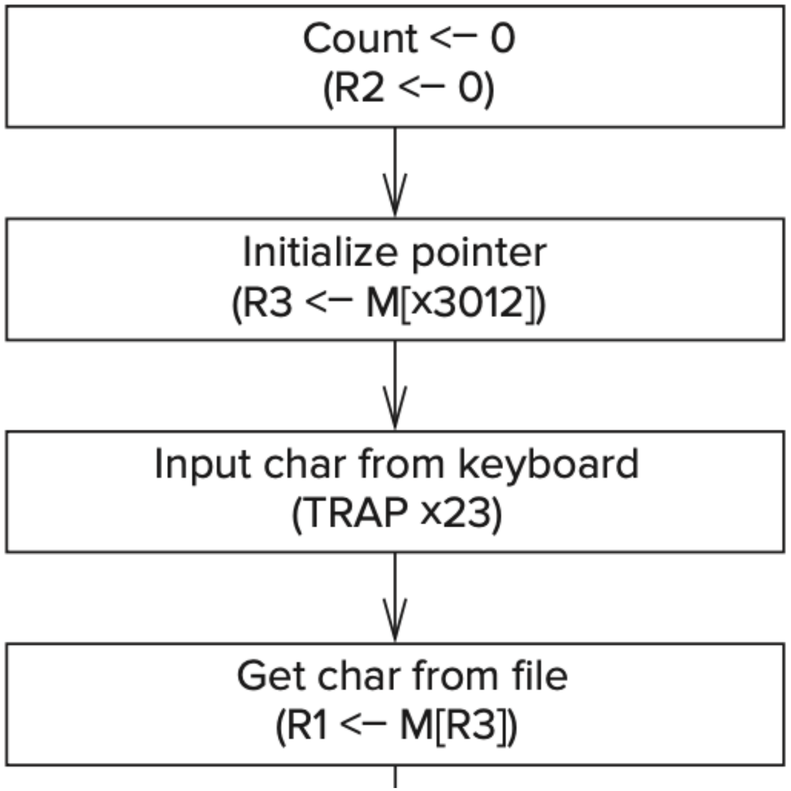
Control flow when breaking down the problem…

- 3 types of control flow…



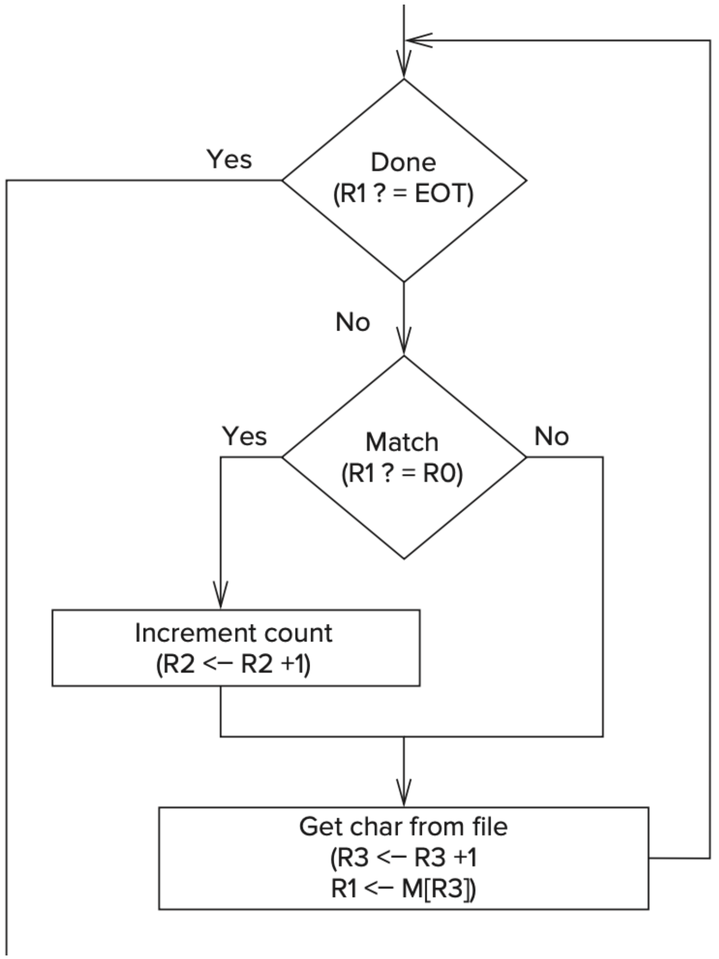
Counting Occurrence of a Character

Part 1: Get Input; Initialize Counter



Counting Occurrence of a Character

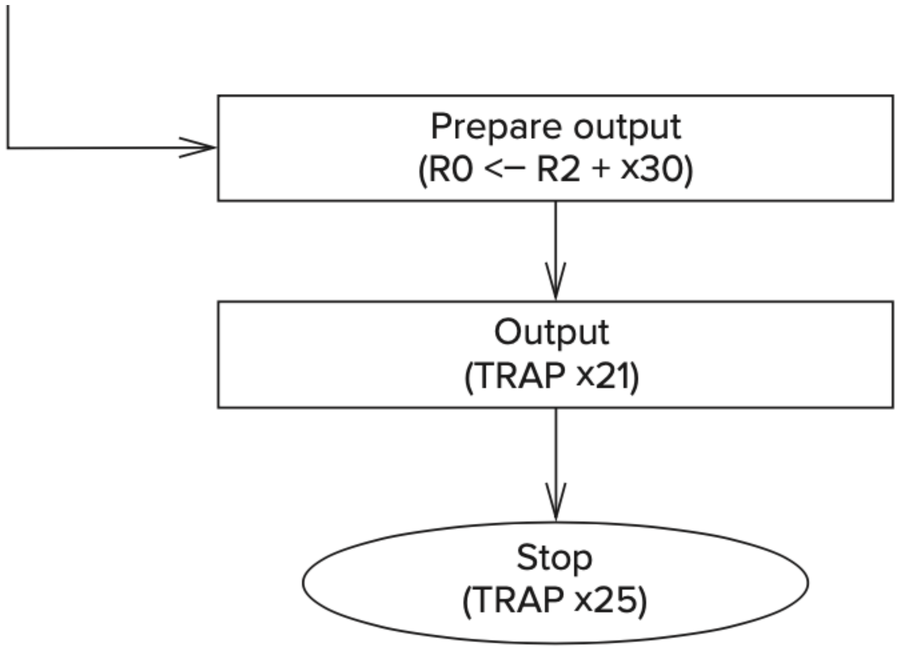
Part 2: Process inputs; counting matches



Counting Occurrence of a Character

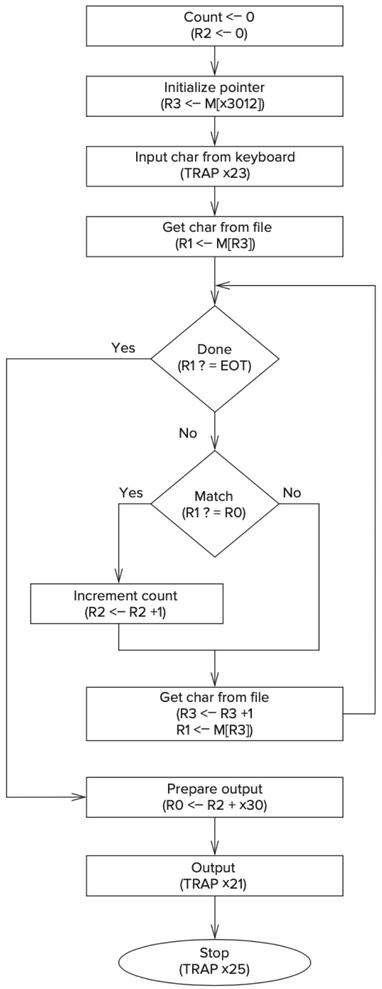
Part 3: Prepare Outputs, Finishing Up

If we reached EOT…



Counting Occurrence of a Character

Big Picture, then assign corresponding resources…



How many registers do we need?

How to call the loops?

How to make branches?

Hazard: Why computers can not run faster despite of parallelism.

Data Hazard

Concurrency of Data

- A data hazard occurs when the pipeline execution must be stalled because one step must wait for another one to complete.
- This comes up when a planned instruction can not execute in the planned clock cycle because the data needed is not yet available.



Data Hazard

4 Types of Data Dependencies

- Read after Write (RAW)
  It is also known as True dependency or Flow dependency. It occurs when the value produced by an instruction is required by a subsequent instruction.

```
ADD R1, --, --;
ADD --, R1, --;
```

- Write after Read (WAR)
  It is also known as anti dependency. These hazards occur when the output register of an instruction is used right after read by a previous instruction.

```
ADD --, R1, --;
ADD R1, --, --;
```

Data Hazard

4 Types of Data Dependencies

- Write after Write (WAW)
  It is also known as output dependency. These hazards occur when the output register of an instruction is used for write after written by previous instruction.

```
ADD R1, --, --;
ADD R1, --, --;
```

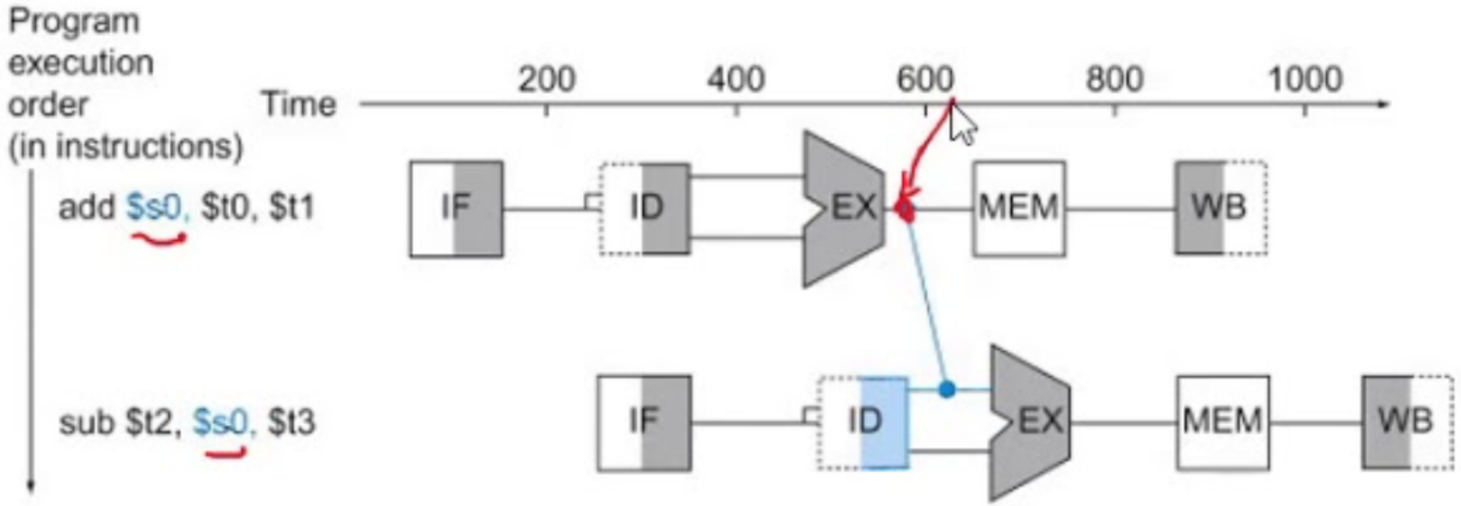
- Read after Read (RAR) :
  It occurs when the instruction both read from the same register.
  Since reading a register value does not change the register value, these Read after Read (RAR) hazards don't cause a problem for the processor.

```
ADD --, R1, --;
ADD --, R1, --;
```

Handling Data Hazards

Change the hardware, change the code, or…

- Forwarding :
  It adds special circuitry to the pipeline. This method works because it takes less time for the required values to travel through a wire than it does for a pipeline segment to compute its result.



Handling Data Hazards

Change the hardware, change the code, or…

- Code reordering :
  We need a special type of software to reorder code. We call this type of software a hardware-dependent compiler.

```
ADD R0 R1 R2
ADD R2 R2 R1
LD  R5 M[x3100]
ADD R2 R0 R5
```

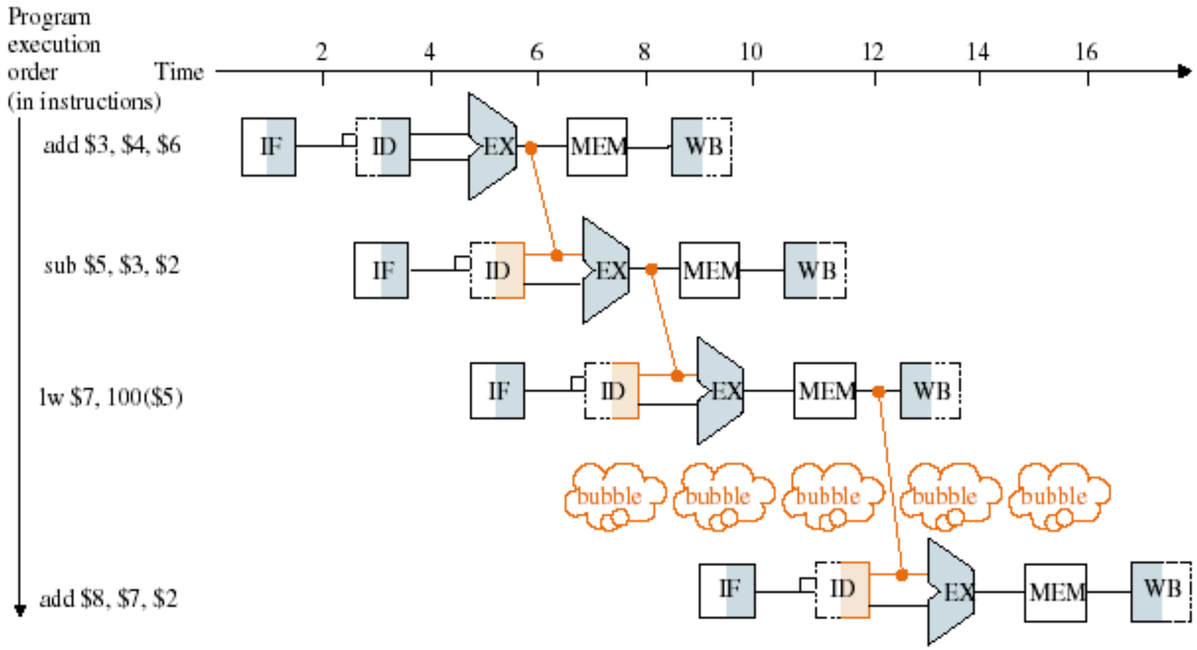


```
ADD R0 R1 R2
LD  R5 M[x3100]
ADD R2 R2 R1
ADD R2 R0 R5
```

Handling Data Hazards

Change the hardware, change the code, or…

- Stall Insertion :
  it inserts one or more stall (no-op instructions) into the pipeline, which delays the execution of the current instruction until the required operand is written to the register file, but this method decreases pipeline efficiency and throughput.



Control Hazard

Time and Effort Lost with Wrong Prediction

- An instruction that is control dependent on a branch cannot be moved in front of the branch, so that the branch no longer controls it and an instruction that is not control dependent on a branch cannot be moved after the branch so that the branch controls it. This will give rise to control hazards.

- Wrong branch predictions will cause two problems if not handled well: exception behavior and handling and preservation of data flow.

```
ADD R2, R3, R4;
LD  R1, R2
BRz R2, #3;
```

Control Hazard

Time and Effort Lost with Wrong Prediction

- Stall until the branch outcome is known or perform the fetch again
- Predict the behavior of branches… better, with…
  - Static prediction by the compiler
  - Dynamic prediction by the hardware:
    - Predict not taken approach
      Assume that the branch is not taken, i.e. the condition will not evaluate to be true
    - Predict taken approach
      Assume that the branch is taken, i.e. the condition will evaluate to be true
    - Delayed branching
      A more effective solution

Pipeline Speed-Up

Don’t panic! More on that later!

$$PipelineSpeedUp = \frac{PipelineDepth}{1 + PipelineStallCPI} = \frac{PipelineDepth}{1 + BranchFrequency * BranchPenalty}$$

Next up…
Quantitative Evaluation on Hazard